

Practical Session 4: LLM Agents and RAG in Finance

Hands-On: ReAct Tracing, Tool Design, and RAG Evaluation

Juan F. Imbet

Paris Dauphine – PSL University

Large Language Models in Finance

Session Overview

Duration: 1 hour

- ① **Recap** (5 min) — PRA loop, ReAct format, memory types, RAGAS metrics
- ② **Problem 1** (15 min) — Trace a ReAct agent on a filing Q&A task
- ③ **Problem 2** (10 min) — Design a tool schema for a financial calculator
- ④ **Case Study** (20 min) — Build and evaluate a minimal RAG pipeline over a mock earnings call corpus; answer three diagnostic questions
- ⑤ **Discussion** (5 min) — When would you add a second agent? When would you add a hook?
- ⑥ **Solutions** (5 min) — Walkthrough of Problems 1 and 2

What you will practise

Tracing agent reasoning steps; writing Pydantic tool schemas; building and evaluating a RAG pipeline; thinking through governance checkpoints.

Recap: PRA Loop and ReAct

PRA loop (π, μ, ϕ) :

- ϕ : observation + memory $\rightarrow$ updated memory
- π : memory $\rightarrow$ distribution over actions (implemented by the LLM)
- μ : action + environment $\rightarrow$ next observation

ReAct trajectory:

$$\tau = (T_1, A_1, O_1, T_2, A_2, O_2, \dots, T_n, \text{FINISH})$$

Each T_i is a natural-language thought. Each A_i is a tool call. Each O_i is the tool's return value.

Key principle: actions ground reasoning in verifiable external data.
 $\hat{f}(t) \neq f(t)$ in general; calling the API returns the exact $f(t)$.

Recap: Memory and RAG Evaluation

Memory types:

Type	Size bound	Persistence
In-context	Context window	Session only
Retrieval (vector DB)	Unbounded	Persistent
Long-term (structured DB)	Unbounded	Persistent

RAGAS metrics:

- **Faithfulness:** claims in answer supported by retrieved context
- **Answer Relevance:** answer semantically addresses the query
- **Context Precision:** retrieved chunks are relevant
- **Context Recall:** ground-truth claims are covered by context

Finance priority: **Faithfulness** first (unfaithful = hallucinated claim = compliance risk). Faithfulness $< 0.85 \rightarrow$ block automated publication.

Problem 1: Trace a ReAct Agent (15 min)

A compliance analyst asks an agent:

"What credit risks does Microsoft disclose in its most recent 10-K, and how do they compare to the risks it disclosed in the 2022 10-K?"

Available tools:

- `edgar_search(ticker, form, limit)` — returns accession numbers and dates
- `edgar_section(accession, section)` — returns text of a named section
- `summarise_risks(text, focus)` — returns structured risk list
- `compare_lists(list_a, list_b)` — returns added, removed, changed items

Your task: write the full ReAct trajectory ($T_1, A_1, O_1, \dots, \text{FINISH}$).

- Include at least 4 steps before the `FINISH` action
- Each thought must justify *why* you choose the next action
- Make up plausible (but fictional) observation values

Problem 2: Design a Financial Tool Schema (10 min)

You are building an LLM agent for a fixed-income desk. Design a Pydantic schema for the following tool:

Tool: `compute_bond_price`

Computes the present value of a bond given its cash flows and yield to maturity. Should support both annual and semi-annual coupon conventions.

Your schema must include:

- Face value (in USD, any positive value)
- Annual coupon rate (as a decimal)
- Years to maturity (positive integer)
- Yield to maturity (as a decimal; annual, not semi-annual)
- Coupon frequency (annual or semi-annual)

For each field: write a `Field(description=...)` with units, valid range, and at least one example value. Use `ge`, `le`, or `gt` constraints.

Bonus: what is the formula for bond price? Add it as a docstring.

Case Study: RAG over Earnings Call Transcripts (20 min)

Setup: You have a mini-corpus of 5 earnings call excerpts (provided as strings). You will build a minimal RAG pipeline and evaluate it using RAGAS-style metrics.

The corpus (fictional, for illustration):

- `apple_q4_2023.txt` — 3 paragraphs, CFO guidance + China exposure
- `microsoft_q2_2024.txt` — 3 paragraphs, Azure growth + AI capex
- `jpmorgan_q1_2024.txt` — 3 paragraphs, NIM guidance + credit losses
- `nvidia_q3_2023.txt` — 3 paragraphs, data centre demand + export controls
- `berkshire_2023.txt` — 3 paragraphs, insurance float + equity portfolio

Queries to evaluate:

- 1 *"What revenue guidance did Apple give for Q1 2024?"*
- 2 *"What did JPMorgan say about credit loss provisions?"*

Case Study Step 1: Chunking and Embedding

```
from sentence_transformers import SentenceTransformer
import numpy as np

# --- Sample corpus (use provided strings in the lab notebook) ---
docs = {
    "apple_q4_2023": "Revenue guidance for Q1 2024 is $89-90B ...",
    # ... add remaining documents
}

# --- Chunk each document into sentences ---
def sentence_chunk(text):
    return [s.strip() for s in text.split('.') if len(s.strip()) > 20]

chunks = []
for doc_id, text in docs.items():
    for sent in sentence_chunk(text):
        chunks.append({"doc_id": doc_id, "text": sent})

# --- Embed all chunks ---
model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode([c["text"] for c in chunks])
```


Case Study Step 2: Retrieve and Generate

```
from sklearn.metrics.pairwise import cosine_similarity

def retrieve(query, k=3):
    q_emb = model.encode([query])
    sims = cosine_similarity(q_emb, embeddings)[0]
    top_k = np.argsort(sims)[::-1][:k]
    return [chunks[i] for i in top_k]

def generate_answer(query, context_chunks, llm_fn):
    context = "\n".join(c["text"] for c in context_chunks)
    prompt = f"""Answer based ONLY on the context below.
    If the answer is not in the context, say "Not found in context."

    Context:
    {context}

    Question: {query}
    Answer: ""
    return llm_fn(prompt)
```

Question B: For query 1 (Apple guidance), which chunks are retrieved ($k = 3$)? Are they all from apple_q4_2023? What does this tell you about metadata filtering?

Case Study Step 3: Evaluate with RAGAS Metrics

For each of the three queries, fill in this table:

Query	Faith.	Ans.Rel.	Ctx.Prec.	Ctx.Rec.
Apple guidance	?	?	?	?
JPM credit losses	?	?	?	?
Export controls	?	?	?	?

Score each metric 0–1 manually using the definitions:

- **Faithfulness:** does each sentence in the answer appear in the context?
- **Answer Relevance:** does the answer address the question?
- **Context Precision:** are the retrieved chunks actually useful?
- **Context Recall:** does the context contain everything needed to answer?

Question C: Which query has the lowest faithfulness? Why? What would you change in the pipeline to improve it?

Discussion: Agent Design Decisions (5 min)

Discuss in pairs or small groups:

When would you add a second agent?

- When the task has two distinct sub-tasks with different tool requirements?
- When you need a critic or compliance checker separate from the generator?
- When latency can be reduced by parallelism?

When would you add a hook?

- When a cross-cutting concern (audit, cost monitoring) must apply to every action?
- When the concern is owned by a different team than the agent developer?
- When you need to enforce a policy without modifying the agent's core logic?

Governance question:

Your earnings call agent produces a structured summary that a junior



Solutions: Problem 1 — ReAct Trajectory

T1 I need Microsoft's most recent 10-K and the 2022 10-K. I will search EDGAR for both.

A1 `edgar_search("MSFT", "10-K", 2)`

O1 `[{acc: "0001564590-24-...", year: 2024}, {acc: "0001564590-22-...", year: 2022}]`

T2 I have both accession numbers. Retrieve risk factors from each filing.

A2 `edgar_section("0001564590-24-...", "Risk Factors")`

O2 `[ 8,000 words of 2024 risk factors text]`

T3 Now retrieve 2022 risk factors.

A3 `edgar_section("0001564590-22-...", "Risk Factors")`

O3 `[ 7,500 words of 2022 risk factors text]`

T4 Summarise credit risks specifically from each set of risk factors.

A4 `summarise_risks(O2, "credit") + summarise_risks(O3, "credit")`

O4 Two structured risk lists.

T5 Compare the two lists to identify what changed.

A5 `compare_lists(risk_list_2024, risk_list_2022)`

Solutions: Problem 2 — Bond Price Tool Schema

```
from pydantic import BaseModel, Field
from typing import Literal

class BondPriceInput(BaseModel):
    """
    Bond price =  $\sum_{t=1}^n C/(1+y)^t + F/(1+y)^n$ 
    where C = coupon payment, y = period yield, n = total periods,
           F = face value.
    """
    face_value: float = Field(
        description="Face value in USD (e.g. 1000.0)", gt=0.0
    )
    annual_coupon_rate: float = Field(
        description="Annual coupon rate as a decimal (e.g. 0.05 for 5%)",
        ge=0.0, le=1.0
    )
    years_to_maturity: int = Field(
        description="Years to maturity (e.g. 10)", gt=0
    )
    yield_to_maturity: float = Field(
        description="Annual YTM as a decimal (e.g. 0.06 for 6%)",
        ge=0.0, le=1.0
    )
```

Key Takeaways

- 1 **ReAct grounds reasoning in evidence.** Each tool-call observation replaces a potentially hallucinated approximation. Thought steps explain *why* each action is chosen.
- 2 **Tool schemas are a first-class design artifact.** Field descriptions propagate into the model's context. Pydantic constraints reduce the error surface.
- 3 **RAG evaluation must be decomposed.** Faithfulness (generation quality) and context recall (retrieval quality) fail for different reasons and require different fixes.
- 4 **Metadata filtering is not optional.** Unconstrained semantic search is noisy for named-entity queries. Add document metadata at ingestion time.
- 5 **Governance patterns match reversibility.** Pre-execution approval for irreversible actions. Post-hoc audit for high-volume, low-stakes tasks.